

Task Table + WCET Evidence

Task	Priority	Period	Role
ISR (bottom half dispatch)	N/A (hardware)	Aperiodic (button press)	Debounce, GPIO19 pulse, hand off via semaphore/notification
Sem wait task	12	Aperiodic (blocks on semaphore)	Wakes on xSemaphoreGiveFromISR, logs latency
Notify wait task	12	Aperiodic (blocks on notify)	Wakes on xTaskNotifyGiveFromISR, logs latency
Load Task A	15	10 ms	Fixed compute fixture — only task that outranks your bottom half
Load Task B	10	20 ms	Fixed compute fixture — cannot preempt bottom half
Load Task C	5	50 ms	Fixed compute fixture — cannot preempt bottom half
Load Task D	2	100 ms	Fixed compute fixture — cannot preempt bottom half

Task A is the only task that can delay the wake progress, B, C, and D are below the bottom half and can only run in the gaps.

WCET Evidence:

Path	n	Min	Max (WCET candidate)	Recurrence of max	Distribution
Notification	51	24 μ s	30 μs	Hit repeatedly — the dominant value across ~35 of 51 presses	Tight, unimodal (24–30 μ s)
Semaphore	51	249 μ s	2192 μs	Hit exactly at presses 13, 14, 17, 26, 37, 39, 40, 44; within 1 μ s (2191) at 10 more presses	Bimodal — low cluster (~249–420 μ s), high cluster (~2040–2192 μ s)

2192 isn't noise because it occurred 8 times across an entire run , so its not a fluke. But because of the wake each bottom half task is taken after the blocking call returns, so it captures how long the task had to wait to actually get schedules and because both tasks sit at equal priority on the same core. Whichtever task the scheduler runs first holds the CPU through its own blocking , which delays the other. The 2192 cluster is consistent with the sem task which is waiting for the notif task's print, the semaphore WCET is measuring the scheduling order + UART contention, not the semaphore.

Used Claude to create all tables